

Introduction to ML Strategy

Why ML Strategy

Maybe we have an application that reach 90% of accuracy, but this doesn't satisfy you, then you get some ideas:

Motivating example



Ideas:

- Collect more data ←
- Collect more diverse training set
- Train algorithm longer with gradient descent
- Try Adam instead of gradient descent
- Try bigger network
- Try smaller network
- Try dropout
- Add L_2 regularization
- Network architecture
 - Activation functions
 - # hidden units
 - ...

Andrew Ng

And the problem is that if you choose poorly, it is entirely possible that you end up spending six months charging in some direction only to realize after six months that that didn't do any good.

For example, I've seen some teams spend literally six months collecting more data only to realize after six months that it barely improved the performance of their system.

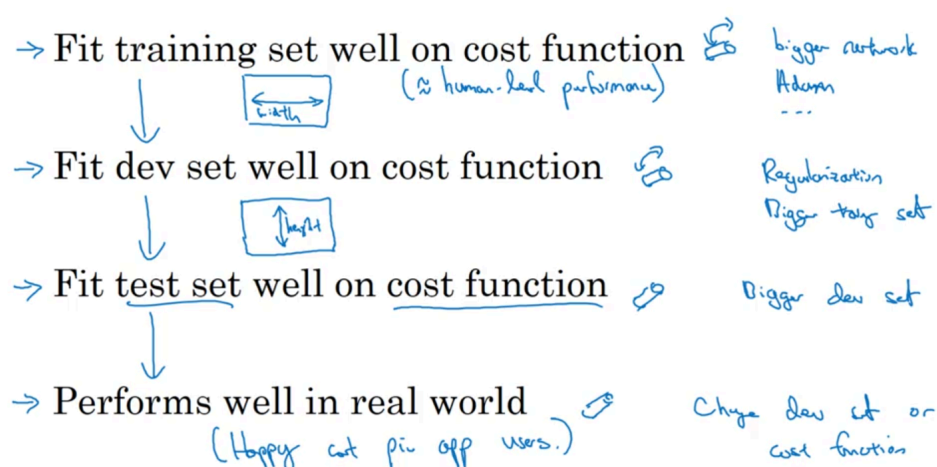
So, assuming you don't have six months to waste on your problem, won't it be nice if you had quick and effective ways to figure out which of all of these ideas and maybe even other ideas, are worth pursuing and which ones you can safely discard. So what I hope to do in this course is teach you a number of strategies, that is, ways of analyzing a machine learning problem that will point you in the direction of the most promising things to try.

Orthogonalization

What is orthogonalization?

- One of the challenges with building machine learning systems is that there's so many things you could try, so many things you could change. Including, for example, so many hyperparameters you could tune. One of the things I've noticed is about the most effective machine learning people is they're very clear-eyed about what to tune in order to try to achieve one effect. This is a process we call orthogonalization.

Chain of assumptions in ML



- Just like when you look at the TV image, it's nice if you can say, my TV image is too wide, so I'm going to tune this knob, or it's too tall, so I'm going to tune that knob, or it's too trapezoidal, so I'm going to have to tune that knob.
- In machine learning, it's nice if you can look at your system and say, this piece of it is wrong. It does not do well on the training set, it does not do well on the dev set, it does not do well on the test set, or it's doing well on the test set but just not in the real world. But figure out exactly what's wrong, and then have exactly one knob, or a specific set of knobs that helps to just solve that problem that is limiting the performance of machine learning system. So what we're going to do this week and next week is go through how to diagnose what exactly is the bottleneck to your system's performance. As well as identify the specific set of knobs you could use to tune your system to improve that aspect of its performance.

Orthogonalization in the machine learning process is a **problem-solving and debugging principle**, not just a linear-algebra concept.

It means **designing your ML system so that each decision or component affects only one aspect of performance**, allowing you to improve things **independently** without causing unintended side effects.

Intuition (simple idea)

| One knob → one effect.

If you turn a knob to fix **training error**, it should not unexpectedly affect **generalization**, **latency**, or **fairness**.

This makes ML systems **easier to debug, tune, and scale**.

Orthogonalization vs non-orthogonal systems

✗ Non-orthogonal (bad design)

- Changing model size affects:
 - Training accuracy
 - Overfitting
 - Inference time
 - Memory usage
 → Hard to know *what* you are fixing

✓ Orthogonal (good design)

- Model capacity → affects **training error**
- Regularization → affects **overfitting**
- More data → affects **generalization**
- Quantization / pruning → affects **latency & memory**

Each change has a **clear, isolated purpose**.

Orthogonalization in the ML workflow

1 Fit training data well

Goal: Low training error

Knobs:

- Bigger model
- Better architecture
- Better optimizer

👉 Should *not* directly affect test error if overfitting is controlled later.

2 Generalize to dev/test data

Goal: Reduce overfitting

Knobs:

- Regularization (L2, dropout)
- Data augmentation
- More training data
- Early stopping

👉 Should *not* break training performance.

3 Match real-world distribution

Goal: Dev/test reflects real usage

Knobs:

- Better dataset collection
- Data cleaning
- Distribution rebalancing

👉 Independent of model architecture.

4 Optimize deployment constraints

Goal: Speed, memory, cost

Knobs:

- Model pruning
- Quantization
- Distillation
- Batch size

👉 Should not change model *correctness*, only efficiency.

Concrete example (image generation / AI models)

Since you work with **Text-to-Image models**:

Problem	Orthogonal fix
Blurry images	Better architecture / loss
Overfitting styles	Data augmentation
Slow inference	Quantization / LoRA
Bad Vietnamese history details	Better dataset curation

Each fix targets **one issue only**.

Why orthogonalization matters in research & practice

- Faster debugging
- Clear ablation studies
- Easier collaboration
- More convincing papers
- Cleaner experiment design

Andrew Ng emphasizes this heavily in **ML system design**.

Short definition (for exams / papers)

Orthogonalization is the practice of structuring machine learning systems so that each design choice affects only one performance aspect, allowing independent optimization and easier debugging.